CS 5433 SP23: Homework 3

Instructor: Ari Juels
TA: James Austgen
Due: 13 April 2026

Policies

Submit your written solutions and code to CMSX.

Integrity. For all assignments, you may make use of published materials, but you must acknowledge all sources, in accordance with the Cornell Code of Academic Integrity. Additionally, you must ensure that you understand the material you are submitting; you must be able to explain your solutions to the course instructor or TA if requested. You must complete this homework assignment *on your own*. You are permitted to use LLMs (e.g., ChatGPT), provided you indicate in code comments where you made use of them.

Problem 1 - Tokens and Simple Smart Contracts

To get our hands dirty with the kinds of smart contracts we've discussed in class on Ethereum, we will create a simple token contract. The directory `ERC20` contains a test skeleton and instructions for installing/running dependencies in the file `README.md`, the version we use here should be $\geq 0.8.0$. We also provide some Solidity code for a basic ERC-20 token, as described in the specification here: `https://github.com/ethereum/ercs/blob/master/ERCS/erc-20.md`. Such tokens are the basis of the "ICO craze" that swept the cryptocurrency community and media some years ago: See, e.g., `https://techcrunch.com/2017/05/23/wtf-is-an-ico/`.

You may also find the Solidity documentation helpful, as the contracts we are writing will be Solidity code: `https://docs.soliditylang.org/en/latest/` (and if you learn by example: `https://docs.soliditylang.org/en/latest/solidity-by-example.html`). Solidity is similar to JavaScript, Java, or C/C++ syntax, and thus should be relatively familiar.

We will design our token on your local filesystem. However, if you do find installing Solidity on your machine to be challenging, you can also use the online IDE Remix, which we will introduce later.

Your task is to enhance our basic ERC-20 contract, provided in `ERC20.sol` as follows:

1. Change the token name to your NetID.

2. This particular ERC-20 token is meant to be pegged to the value of 1000 wei. A wei is the base unit of currency in the Ethereum system; see `https://eth-converter.com/extended-converter.html` for a conversion table. A new function `deposit` is included that permits a caller to generate and obtain ownership of a fresh token for 1000 wei. Given `msg.value` of x wei, the function generates $\tau = \lfloor x/1000 \rfloor$ tokens, assigns them to `msg.sender`, and refunds $x - 1000 \times \tau$ tokens to `msg.sender` (the remaining balance). Notice the use of the `msg.value` keyword, which provides the contract access to the current message value in wei.

   Your task is to implement the corresponding `withdraw` function, allowing senders to redeem each token for the 1000 wei they deposited to obtain it. A placeholder is provided in the given file.

One way to test your contract is with the provided tests, which you can run by `python3 run_tests.py` (see `README.md` for dependency installation instructions - you will have to install the Solidity Compiler, `solc`, to your machine). On top of that, you can also manually test your contract by trying a deposit and withdrawal on a live Ethereum test network (e.g., Sepolia or Hoodi). Once you have completed the contract, you must deploy it to the Hoodi network. Instructions for interacting with the testnet are below. You can find screenshots for these steps in the Metamask and Remix Setup Screenshots appendix.

1. Install Metamask on Firefox or Chrome. Follow the provided steps to set up a Metamask account. **Important:** Add the Hoodi testnet as a network following the steps in Figure 1. Switch Metamask to Hoodi Testnet using the dropdown menu at the top right of the Metamask interface.

2. Get some Hoodi Ether. You can obtain Hoodi Ether from one of these faucets listed here: `https://faucetlink.to/hoodi`.

   Hoodi is an Ethereum test network. You DO NOT need to pay money or purchase any actual Ether tokens.

3. Use `https://remix.ethereum.org/` to compile and deploy your contract.

   (a) To compile, select "Open Files" on the home page and select your ERC20.sol file. On the left hand side, you should see several tabs. Go to the tab that says "Solidity compiler" when you hover your mouse over it (this should be the third one down). Click "Compile ERC20.sol" to compile. If successful, you should see a green checkmark appear at the tab icon.

   (b) Next you will deploy your contract. Go to the tab that says "Deploy & run transactions" (fourth tab down). Switch the environment to "Injected Provider - Metamask" and select "MyToken - ERC20.sol" in the drop down above the Deploy button. **Important**: Make sure you have your "ERC20.sol" file open and not the *Home* tab open for deployment. Now click the Deploy button. A Metamask window should appear in your browser (if not, click the Metamask icon). Follow all on-screen prompts to confirm. After finishing, a link to the deployment transaction will appear in the Remix console (you can find this at the bottom of the Remix page). Once the transaction is included in a block, the contract address will be shown in that link in the "To" field. You can also find the contact address on the left side of Remix under "Deployed Contracts". To view your contract on the public testnet, you can enter the address of the contract at `https://eth-hoodi.blockscout.com/` or `https://hoodi.etherscan.io/` and see all relevant transactions.

   Tip: Before deploying to Hoodi, you can test out deploying and running your contract entirely within Remix using the "Remix VM (Osaka)" environment.

Write your code in the provided **ERC20.sol** file and your deployed contract address (not the transaction hash of contract creation) in **ERC20_addr.txt** file. The txt file should contain one single line of a contract address such as `0x0BE207E608Af340c3B238901120A0fAa2bbc0E9C`. Don't put anything else in it.

Problem 2 - Gaming Contracts

Ethermon (`https://web.archive.org/web/20210815152829/https://docs.ethermon.io/`) is a Pokémon-like game played via an Ethereum contract. Ethermon players catch or buy monsters, and then build their monsters' skills by means of training in a "gym" or by battling with other monsters. The outcome of a battle is determined by randomness derived from the blockchain using this function:

```
1 function getRandom(uint8 maxRan, uint8 index, address priAddress) constant public returns(
    uint8) {
2     uint256 genNum = uint256(block.blockhash(block.number-1)) + uint256(priAddress);
3     for (uint8 i = 0; i < index && i < 6; i ++) {
4         genNum /= 256;
5     }
6     return uint8(genNum % maxRan);
7 }
```

Listing 1: Ethermon Randomness Source

`block.blockhash` is used to return the hash of the last block, which is converted to integer form. Some determinsitic post-processing then happens according to a randomness index and the address of e.g., the monster that is currently battling, ensuring uniqueness inside the contract.

We've created a simplified version of Ethermon called EthermonLite, packaged with the homework in `p2/EthermonLite.sol`, that determines battle outcomes in a similar way. EthermonLite allows users to battle the house, and naturally biases battles *heavily* in favor of the house, represented as a monster called the Ogre. EthermonLite is running on the Hoodi testnet at address `0xa4672a6345819F4df9aFf2F9822a22C3a58b6A10` and can be viewed on Blockscout or Etherscan. The contract has been verified on both platforms for easier interaction from your browser.

Unfortunately, the method used for randomness generation in EthermonLite is vulnerable to manipulation. Specifically, notice that the outcome depends on both the previous block hash and the challenger's full name (the challenger's base name + space character + the challenger's title).

```
// XOR the previous block hash with the SHA-256 hash of the challenger's full name
uint dice = uint(blockhash(block.number - 1));
uint challengerDice = dice ^ uint(sha256(bytes(getFullName(challenger))));

// Challenger wins only if challengerDice mod battleRatio == 0
if (challengerDice % battleRatio == 0) {
    monsters[challenger].wins += 1;
    monsters[Ogre].losses += 1;
    challengerWins = true;
}
```

Listing 2: EthermonLite Battle Code

Your task is to create a monster and hack EthermonLite so that your monster repeatedly defeats the Ogre.

You must submit:

1. The source code for a contract you used to accomplish your exploit. Use the template **WinBattle.sol** we provide in the `entropy` directory to get started.

2. The Ethereum address monsterAddress of a monster on the Hoodi network that has 50 or more wins and 0 losses on our deployed contract, and whose base name (`monsters[monsterAddress].name`) is your NetID. An example of such a monster's address on-chain for the provided contract is `0x5F5E4c7d208a7b798CA99bD55A1Ef62B9Bdddc14`. Put your winning address into the file **WinBattle_addr.txt**.

Hints:

- Instead of the obvious way of having an account own your monster, have a *contract* own your monster. Remember, each contract has an address and can make calls to other contracts, hold tokens, contain a constructor that performs setup, etc.! Launch your own local instance of EthermonLite in Remix and play with these monster-owning contracts, but remember, you must do your final testing on-chain. When doing your on-chain transactions, make sure to use enough gas. You may need to increase the gas limit of your transactions to 700,000 or more, depending on your implementation.

- To interact with the EthermonLite contract from within your contract, you can use this: `IEthermonLite ethermon = IEthermonLite(address(`EthermonLite address`));`

- Check out the transactions that made the working example we provided work; you could always try to reverse engineer their EVM bytecode, but it will probably be easier to write your own code. If you do choose the reverse-engineering route, please try to understand *why* the code works and what this means for why randomness in smart contracts is difficult.

- If you need a way to make strings, you can make use of the included `StringTools.makeString(number)` function. You are not required to use this function, and there are alternative ways of defeating the Ogre.
- To check the number of wins and losses your challenger's address has, you can use the "Read Contract" interface on block explorers (such as Blockscout) and enter the challenger's address into the "getNumWins" and "getNumLosses" functions.

Write your code in **WinBattle.sol** file, and your monster's address in a new **WinBattle_addr.txt** file. Again, don't put anything else in the text file.

Problem 3 - What Anonymity?

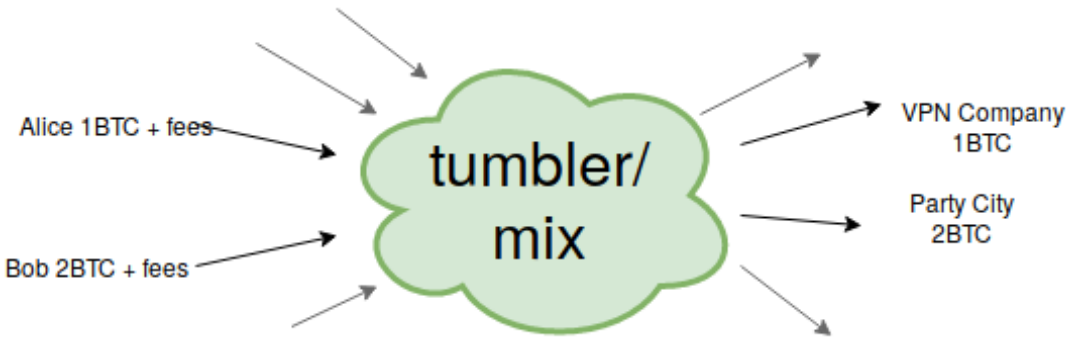
(3a) - Bitcoin

As observed in class, public blockchains like Bitcoin and Ethereum do not provide full anonymity, but instead offer a weaker form of privacy. This limitation is the reason for use of fresh change addresses in Bitcoin transactions, and has also given rise to what are called "mixers" or "tumblers." (See, e.g., `https://en.wikipedia.org/wiki/Cryptocurrency_tumbler`.)

A mix is a service that ingests a set of outputs (BTC) from a set of addresses $A_1, ..., A_n$ and redistributes them to a fresh set of addresses $B_1, ..., B_m$. An observer of a mix's on-chain transactions does not explicitly learn a correspondence between incoming and outgoing addresses.

Mixes can in principle enhance user privacy. For example, suppose that Alice, Bob, and Charlie respectively own $\texttt{addr}_A, \texttt{addr}_B$, and $\texttt{addr}_C$, each with 1 BTC, and this ownership is known to an adversary. If Alice, Bob, and Charlie want to conceal their ownership of BTC, they can send 1 BTC into a mix from their respective addresses $\texttt{addr}_A, \texttt{addr}_B$, and $\texttt{addr}_C$, and have the mix send 1 BTC each to $\texttt{addr}_D$, $\texttt{addr}_E$, and $\texttt{addr}_F$, fresh addresses also controlled respectively by Alice, Bob, and Charlie. If the order of the outputs to these three addresses is randomly permuted in the UTXOs created by the mix, the adversary will be unable to tell if $\texttt{addr}_D$ belongs to Alice, Bob, or Charlie. Consequently, in observing a transaction from $\texttt{addr}_D$, the adversary will be unable to tell which of the three players spent the money. (Note: We are disregarding transaction fees in the example here.)

Tumblers are a type of mix, as is CoinJoin, where users perform a series of transactions together with their coins in a decentralized protocol. Unlike CoinJoin, for most tumblers, users send all money to a centralized service which internally mixes their money together. All users are paid out with a random UTXO held by the mixer, that comes from some other user, breaking the link between the funds on-chain to all but the operator of the tumbler. The operation of both is roughly summarized by the below diagram, which shows Alice mixing a 1BTC payment to her VPN company with Bob's 2BTC Party City payment. Whether a tumbler or a mix is used, the correspondence between inputs and outputs is hidden. In the case of a tumbler, some delay may also be added between the two transactions to prevent timing attacks, and some random fee is charged to increase anonymity. Mixes also charge fees that can potentially be randomized.



Law enforcement and tax collection agencies have employed companies such as Chainalysis (`https://www.chainalysis.com/`) to identify illegal activities such as tax evasion. Mixers and tumblers can make their task more difficult.

In practice, however, mixing or tumbling can offer weaker than ideal privacy. First, as above, players may send unequal amounts into a mix or tumbler. Second, a mix or tumbler may be used to conceal payments, rather than just impart greater privacy to an existing set of players. This usage may constrain the choice of output values emitted by the mix. For example, if Alice is paying a service exactly 1 BTC, then a subset of outputs must sum to 1 BTC. Often goods and services involve payments in round amounts (e.g., 0.1 BTC or the BTC equivalent of $100 USD), making it easier to correlate inputs and outputs.

Your task in this exercise is to partially deanonymize a collection of real tumbler operations observed in Bitcoin.

Consider the following input addresses to a series of tumblers:

1. 1MVXpgczazLvbtS8Nfp9v3Qpj4d8pUNXQM (Grams Helix)
2. 135g5Es7VXvbaAkwzguv7q7xaSSTifav5H (Bitcoin Fog)
3. 1GcZjZnfQUCs9L9RoAFLdd8YET2WQWrDAz (CoinCloud)
4. 1KGhtebk4Nr2zZSn2NaFepeNF6KyjxpPJZ (PenguinMixer)

For each of the following output addresses, indicate the corresponding input that is part of the same mix operation (1.-4.):

- 18RwKzXtL5YGvFwa9BHrPRvqXLkdYWsGfp: _____
- 1MTbp4bFftessrbTTpM5SC5Ap1iKaMHrM7: _____
- 1BCaztysy2paguXjuC8c652vckNMks69ce: _____
- 13MUZ1Qk36LqExdcSRDZCxNRP1pcz1b5mT: _____

You can use blockchain.com to view the transactions on a given address; for example `https://www.blockchain.com/explorer/addresses/btc/13MUZ1Qk36LqExdcSRDZCxNRP1pcz1b5mT`.

Write your answer in **btc-solution.txt**.

(3B) - Ethereum

Some modern privacy-enhancing tools contain features which restrict the ability of malicious actors to use the service. Privacy Pools is one such example. Unlike the Bitcoin tumblers, the Privacy Pools platform runs on Ethereum smart contracts and does not depend on a central party to custody funds. However, Privacy Pools does require all deposits to be approved by a central party; deposits which are not approved (such as ones from malicious origin) may withdraw their funds from the smart contract back to the deposit address and do not gain any privacy protection.

Users with approved deposits may withdraw from the pool to a fresh address by submitting proof to the pool smart contract that they know a secret which unlocks an unclaimed deposit. This proof does not reveal which deposit is being used, and a "relayer" can send this transaction on behalf of the user (who often has no funds at the fresh withdrawal address to do this anyway).

Privacy Pools lets you make multiple withdrawals from the same deposit. For example, if Alice deposits 3 ETH to the pool from $\texttt{addr}_A$, she can withdraw 1 ETH to $\texttt{addr}_B$ and 2 ETH to $\texttt{addr}_C$. If other users are depositing similar amounts, Alice's withdrawals may have plausibly come from other deposits.

Despite being relatively new, Privacy Pools can provide weaker privacy similar to the Bitcoin tumblers if used incorrectly.

In this part of the exercise, you will deanonymize Privacy Pools withdrawals from the same depositor.

Consider the following Ethereum address, which made two deposits to an ETH privacy pool:
`0xA31463942c8b79af930E9F3c8E932D49FdAa8fBc`

Your task is to determine the **single address** where the funds from these two deposits were withdrawn from the pool, and explain how you did so. The address to report is the one which receives the majority of the funds from the "Privacy Pools: Deposit" contract within a `relay` transaction, which withdraws funds from the privacy pool.

Hints:

- The internal transactions which actually move funds to and from the privacy pool smart contract (for all users) are visible on the block explorer here:
  `https://etherscan.io/txsInternal?a=0xF241d57C6DebAe225c0F2e6eA1529373C9A9C9fB&ps=100`
- Deposit transactions move funds from the Deposit contract to the Simple Pool contract. Withdrawal transactions go the opposite direction.
- Start by locating the two deposits on the internal transactions page.

Write your answer in **eth-solution.txt**: on the first line, write the single address you found. On the second line, (1) write a brief explanation of how you were able to identify the withdrawal address, and (2) answer this question: What should the depositor have done differently to retain privacy?

Problem 4 - Survey

- How long did it take you to complete this assignment (in hours)?
- Did you find the homework easy, appropriately difficult, or too difficult?
- Did you feel there was too much coding, the appropriate amount of coding, or not enough coding?

Please submit your answers in `survey.txt` (a one liner is enough). Any other feedback on the homework or class logistics is appreciated!

Submission Instructions

Directory Structure:

```
hw3-<NetID>
├── p1
│   ├── ERC20.sol
│   └── ERC20_addr.txt
├── p2
│   ├── WinBattle.sol
│   └── WinBattle_addr.txt
├── p3
│   ├── btc-solution.txt
│   └── eth-solution.txt
└── p4
    └── survey.txt
```

Metamask and Remix Setup Screenshots

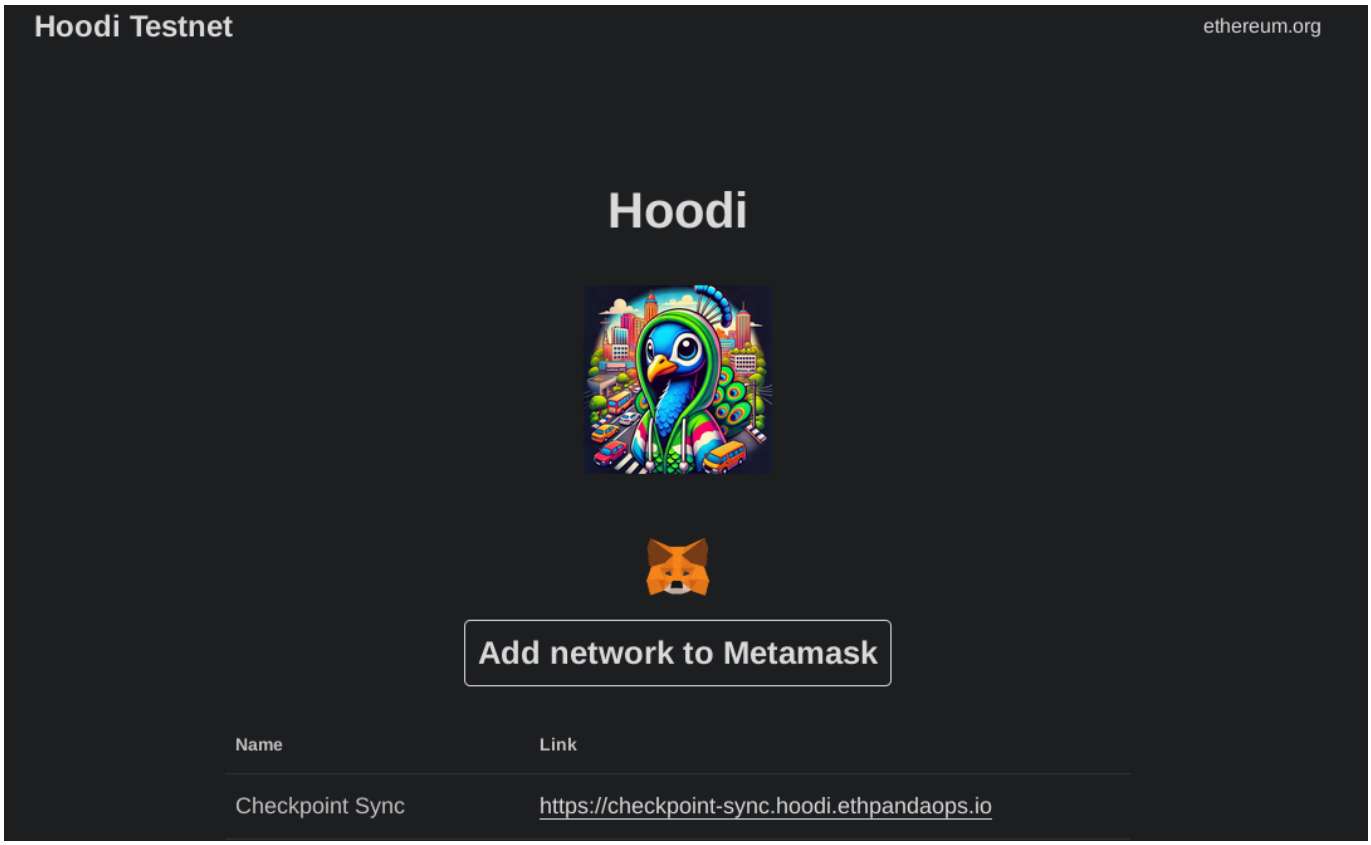


Figure 1: Add the Hoodi Network to your Metamask wallet by clicking "Add network to Metamask" at `https://hoodi.ethpandaops.io/`

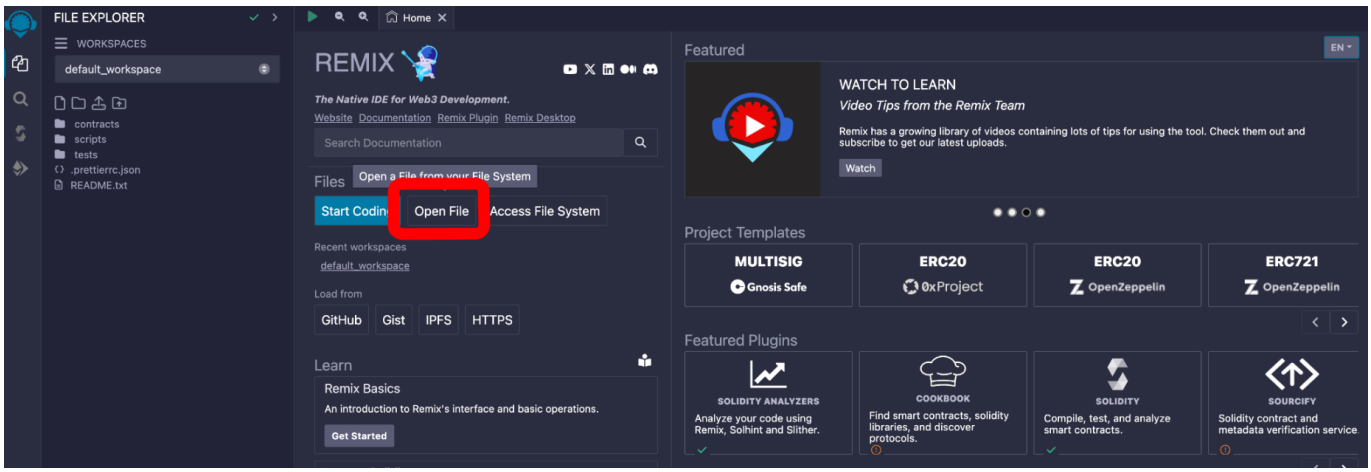


Figure 2: At `https://remix.ethereum.org`, you can load a Solidity source file from your computer by selecting the "Open File" button.

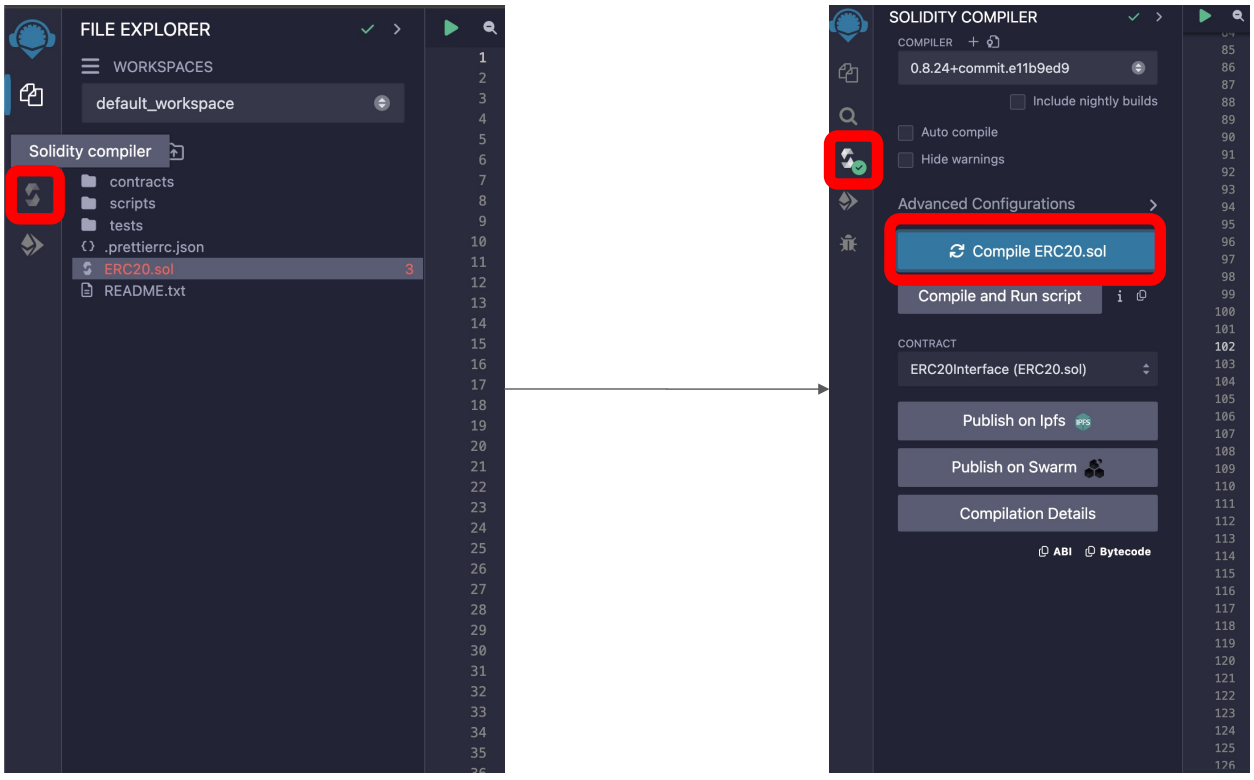


Figure 3: To compile a Solidity smart contract in Remix, make sure you have the smart contract open on the right side of the page. Click the "Solidity compiler" icon on the left panel, and then click the blue "Compile" button. Bonus tip: to save gas on both deployment and during execution, turn on the optimizer under "Advanced Configurations."

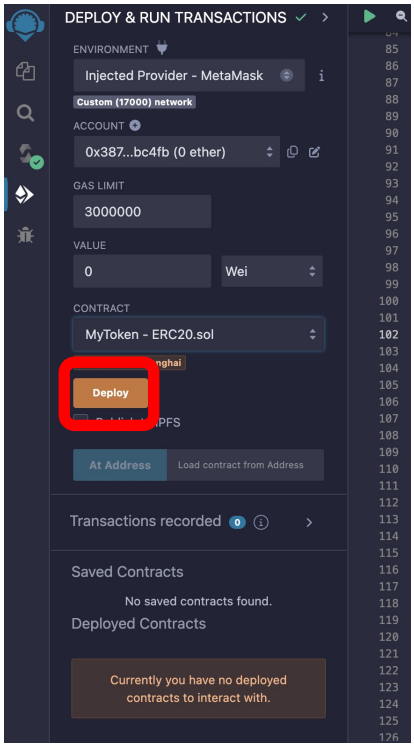


Figure 4: To deploy a smart contract on Hoodi, make sure you change the environment to "Injected Provider" and connect your Metamask wallet to Remix. Then, you can press the Deploy button to publish your smart contract. Make sure the correct contract is selected.